

OBJECT ORIENTED PROGRAMMING

UNIT - II

**AI303PC: OBJECT ORIENTED PROGRAMMING
USING JAVA**

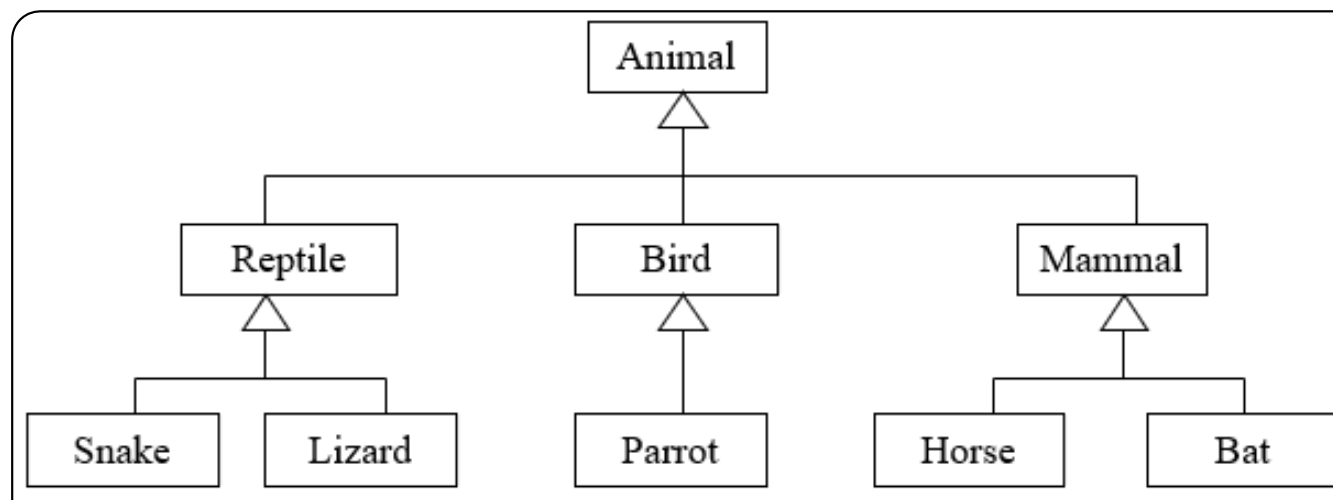


HIERARCHICAL ABSTRACTION

- An essential element of object-oriented programming is *abstraction*.
- Humans manage complexity through abstraction. For example, people do not think of a car as a set of tens of thousands of individual parts. They think of it as a well-defined object with its own unique behavior.
- This abstraction allows people to use a car without being overwhelmed by the complexity of the parts that form the car. They can ignore the details of how the engine, transmission, and braking systems work.
- Instead, they are free to utilize the object as a whole.

CLASS HIERARCHY

- A child class of one parent can be the parent of another child, forming *class hierarchies*
- At the top of the hierarchy, there's a default class called *Object*.



CLASS HIERARCHY

- Good class design puts all common features as high in the hierarchy as reasonable
- The class hierarchy determines how methods are executed
- inheritance is transitive
 - An instance of class Parrot is also an instance of Bird, an instance of Animal, ..., and an instance of class Object

BASE CLASS OBJECT

- In Java, all classes use inheritance.
- If no parent class is specified explicitly, the base class **Object** is implicitly inherited.
- All classes defined in Java, is a child of **Object** class, which provides minimal functionality guaranteed to be common to all objects.

BASE CLASS OBJECT CONTD...

- **Methods defined in Object class are;**
 - 1. equals(Object obj)** Determine whether the argument object is the same as the receiver.
 - 2. getClass()** Returns the class of the receiver, an object of type Class.
 - 3. hashCode()** Returns a hash value for this object. Should be overridden when the equals method is changed.
 - 4. toString()** Converts object into a string value. This method is also often overridden.

BASE CLASS

- 1) a class obtains variables and methods from another class
- 2) the former is called subclass, the latter super-class (Base class)
- 3) a sub-class provides a specialized behavior with respect to its super-class
- 4) inheritance facilitates code reuse and avoids duplication of data

BASE CLASS

- One of the pillars of object-orientation.
- A new class is derived from an existing class:
 - 1) existing class is called super-class
 - 2) derived class is called sub-class
- A sub-class is a specialized version of its super-class:
 - 1) has all non-private members of its super-class
 - 2) may provide its own implementation of super-class methods
- Objects of a sub-class are a special kind of objects of a super-class.

EXTENDS

- Is a keyword used to inherit a class from another class.
- Allows to extend from only one class
- **One baseobj;**// base class object.
- super class object **baseobj** can be used to refer its sub class objects.
- For example, **Two subobj=new Two;**
- **Baseobj=subobj** // now its pointing to sub class

```
class One{  
    int x = 10;  
}
```

```
class Two extends One{  
    int y = 20;  
}
```

SUBCLASS, SUBTYPE AND SUBSTITUTABILITY

- A subtype is a class that satisfies the principle of substitutability.
- A subclass is something constructed using inheritance, whether or not it satisfies the principle of substitutability.
- The two concepts are independent. Not all subclasses are subtypes, and (at least in some languages) you can construct subtypes that are not subclasses.
- Substitutability is fundamental to many of the powerful software development techniques in OOP.
- The idea is that, declared a variable in one type may hold the value of different type.
- Substitutability can occur through use of inheritance, whether using **extends**, or using **implements** keywords.

SUBCLASS, SUBTYPE AND SUBSTITUTABILITY

- When new classes are constructed using inheritance, the argument used to justify the validity of substitutability is as follows;
 - Instances of the subclass must possess all data fields associated with its parent class.
 - Instances of the subclass must implement, through inheritance at least, all functionality defined for parent class. (Defining new methods is not important for the argument.)
 - Thus, an instance of a child class can mimic the behavior of the parent class and should be *indistinguishable* from an instance of parent class if substituted in a similar situation.

SUBCLASS, SUBTYPE AND SUBSTITUTABILITY

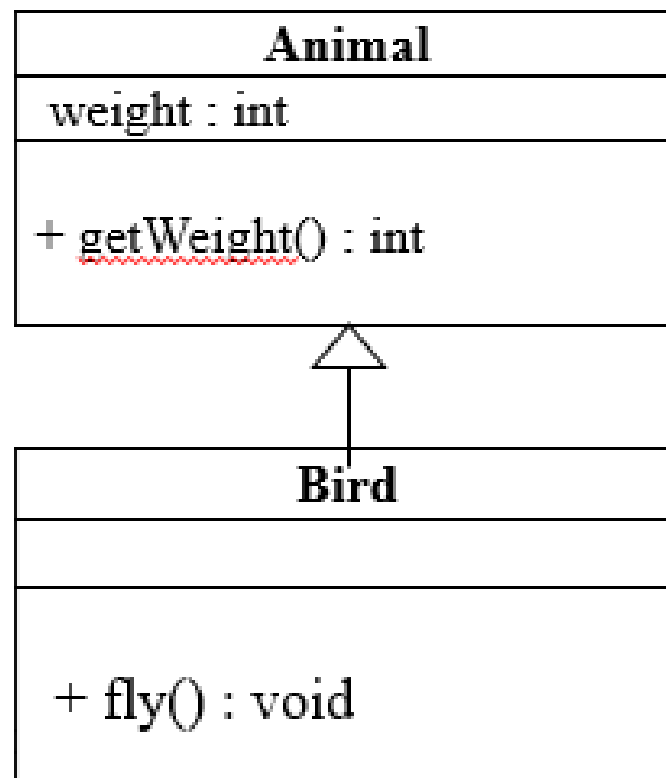
- The term subtype is used to describe the relationship between types that explicitly recognizes the principle of substitution. A type B is considered to be a subtype of A if an instances of B can legally be assigned to a variable declared as of type A.
- The term subclass refers to inheritance mechanism made by extends keyword.
- Not all subclasses are subtypes. Subtypes can also be formed using interface, linking types that have no inheritance relationship.

SUBCLASS

- Methods allows to reuse a sequence of statements
- *Inheritance* allows to reuse classes by deriving a new class from an existing one
- The existing class is called the *parent class*, or *superclass*, or *base class*
- The derived class is called the *child class* or *subclass*.
- As the name implies, the child inherits characteristics of the parent(i.e the child class *inherits* the methods and data defined for the parent class

SUBTYPE

- Inheritance relationships are often shown graphically in a *class diagram*, with the arrow pointing to the parent class.



SUBSTITUTABILITY (DERIVING SUBCLASSES)

- In Java, we use the reserved word `extends` to establish an inheritance relationship

```
class Animal{  
    // class contents  
    int weight;  
    public void int getWeight() {...}  
}  
-----  
class Bird extends Animal{  
    // class contents  
    public void fly() {...};  
}
```

DEFINING METHODS IN THE CHILD CLASS: OVERRIDING BY REPLACEMENT

- A child class can *override* the definition of an inherited method in favor of its own
 - that is, a child can redefine a method that it inherits from its parent
 - the new method must have the same signature as the parent's method, but can have different code in the body
- In java, all methods except of constructors override the methods of their ancestor class by replacement. E.g.:
 - the Animal class has method eat()
 - the Bird class has method eat() and Bird extends Animal
 - variable *b* is of class Bird, i.e. Bird *b* = ...
 - *b.eat()* simply invokes the eat() method of the Bird class
- If a method is declared with the `final` modifier, it cannot be overridden

FORMS OF INHERITANCE

Inheritance is used in a variety of way and for a variety of different purposes .

- Inheritance for Specialization
- Inheritance for Specification
- Inheritance for Construction
- Inheritance for Extension
- Inheritance for Limitation
- Inheritance for Combination

One or many of these forms may occur in a single case.

FORMS OF INHERITANCE

(-INHERITANCE FOR SPECIALIZATION-)

- Most commonly used inheritance and sub classification is for specialization.
- Always creates a subtype, and the principles of substitutability is explicitly upheld.
- It is the most ideal form of inheritance.
- An example of subclassification for specialization is:

```
public class PinBallGame extends Frame {  
    // body of class  
}
```


SPECIALIZATION

- By far the most common form of inheritance is for specialization.
 - Child class is a specialized form of parent class
 - Principle of substitutability holds
- A good example is the Java hierarchy of Graphical components in the AWT:
 - Component

- Label
- Button
- TextComponent
 - TextArea
 - TextField
- CheckBox
- ScrollBar

SPECIALIZATION

- By far the most common form of inheritance is for specialization.
 - Child class is a specialized form of parent class
 - Principle of substitutability holds
- A good example is the Java hierarchy of Graphical components in the AWT:
 - Component

- Label
- Button
- TextComponent
 - TextArea
 - TextField
- CheckBox
- ScrollBar

FORMS OF INHERITANCE

(-INHERITANCE FOR SPECIFICATION-)

- This is another most common use of inheritance. Two different mechanisms are provided by Java, interface and abstract, to make use of subclassification for specification. Subtype is formed and substitutability is explicitly upheld.
- Mostly, not used for refinement of its parent class, but instead is used for definitions of the properties provided by its parent.

```
class FireButtonListener implements ActionListener {  
    // body of class  
}  
class B extends A {  
    // class A is defined as abstract specification class  
}
```

SPECIFICATION

- The next most common form of inheritance involves specification. The parent class specifies some behavior, but does not implement the behavior.
- Child class implements the behavior
- Similar to Java interface or abstract class
- When parent class does not implement actual behavior but merely defines the behavior that will be implemented in child classes
- Example, Java 1.1 Event Listeners:
 - ActionListener, MouseListener, and so on specify behavior, but must be subclassed.

FORMS OF INHERITANCE

(-INHERITANCE FOR CONSTRUCTION-)

- Child class inherits most of its functionality from parent, but may change the name or parameters of methods inherited from parent class to form its interface.
- This type of inheritance is also widely used for code reuse purposes. It simplifies the construction of newly formed abstraction but is not a form of subtype, and often violates substitutability.
- Example is ***Stack*** class defined in Java libraries.

CONSTRUCTION

- The parent class is used only for its behavior, the child class has no *is-a* relationship to the parent.
 - Child modify the arguments or names of methods
- An example might be subclassing the idea of a *Set* from an existing *List* class.
 - Child class is not a more specialized form of parent class; no substitutability

FORMS OF INHERITANCE

(-INHERITANCE FOR EXTENSION-)

- Subclassification for extension occurs when a child class only adds new behavior to the parent class and does not modify or alter any of the inherited attributes.
- Such subclasses are always subtypes, and substitutability can be used.
- Example of this type of inheritance is done in the definition of the class Properties which is an extension of the class HashTable.

GENERALIZATION OR EXTENSION

- The child class generalizes or extends the parent class by providing more functionality
- In some sense, the opposite of subclassing for specialization
- The child doesn't change anything inherited from the parent; it simply adds new features
- Often used when we cannot modify the existing base parent class
- Example, ColoredWindow inheriting from Window
- Add additional data fields
- Override window display methods

FORMS OF INHERITANCE

(-INHERITANCE FOR LIMITATION-)

- Subclassification for limitation occurs when the behavior of the subclass is smaller or more restrictive than the behavior of its parent class.
- Like subclassification for extension, this form of inheritance occurs most frequently when a programmer is building on a base of existing classes.
- Is not a subtype, and substitutability is not proper.

LIMITATION

- The child class limits some of the behavior of the parent class.
- Example, you have an existing List data type, and you want a Stack
- Inherit from List, but override the methods that allow access to elements other than top so as to produce errors.

FORMS OF INHERITANCE

(-INHERITANCE FOR COMBINATION-)

- This type of inheritance is known as multiple inheritance in Object Oriented Programming.
- Although Java does not permit a subclass to be formed by inheritance from more than one parent class, several approximations to the concept are possible.
- Example of this type is Hole class defined as:

```
class Hole extends Ball implements PinBallTarget{  
    // body of class  
}
```

COMBINATION

- Two or more classes that seem to be related, but its not clear who should be the parent and who should be the child.
- Example: Mouse and TouchPad and JoyStick
- Better solution, abstract out common parts to new parent class, and use subclassing for specialization.

SUMMARY OF FORMS OF INHERITANCE

- **Specialization.** The child class is a special case of the parent class; in other words, the child class is a subtype of the parent class.
- **Specification.** The parent class defines behavior that is implemented in the child class but not in the parent class.
- **Construction.** The child class makes use of the behavior provided by the parent class, but is not a subtype of the parent class.
- **Generalization.** The child class modifies or overrides some of the methods of the parent class.

SUMMARY OF FORMS OF INHERITANCE

- **Extension.** The child class adds new functionality to the parent class, but does not change any inherited behavior.
- **Limitation.** The child class restricts the use of some of the behavior inherited from the parent class.
- **Variance.** The child class and parent class are variants of each other, and the class-subclass relationship is arbitrary.
- **Combination.** The child class inherits features from more than one parent class. This is multiple inheritance and will be the subject of a later chapter.

THE BENEFITS OF INHERITANCE

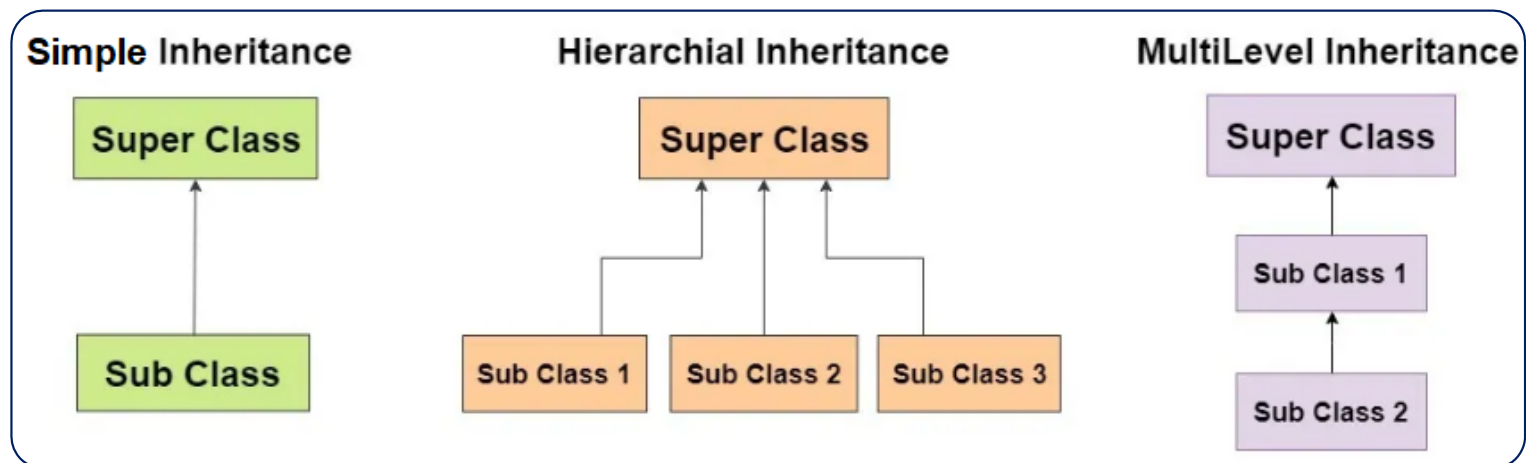
- Software Reusability (among projects)
- Increased Reliability (resulting from reuse and sharing of well-tested code)
- Code Sharing (within a project)
- Consistency of Interface (among related objects)
- Software Components
- Rapid Prototyping (quickly assemble from pre-existing components)
- Polymorphism and Frameworks (high-level reusable components)
- Information Hiding

THE COSTS OF INHERITANCE

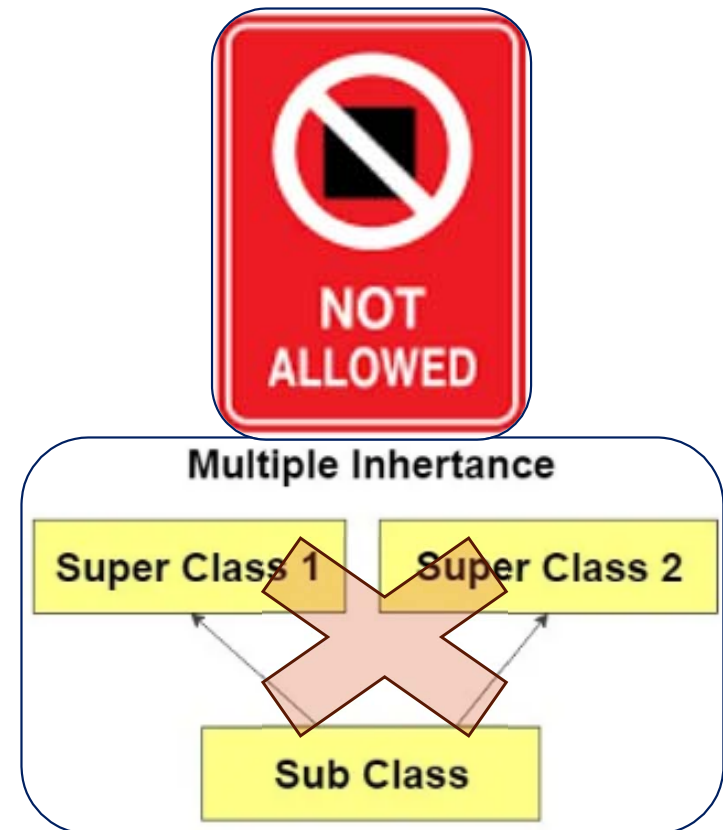
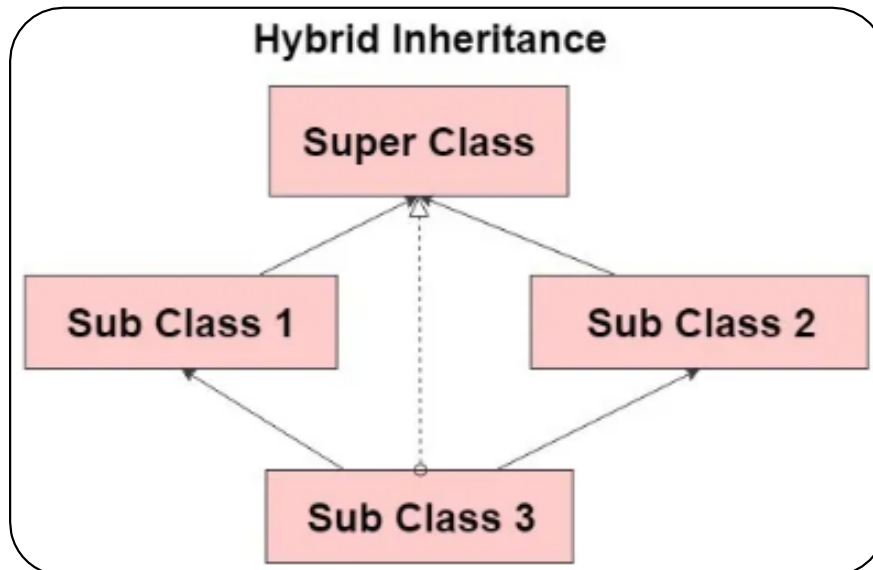
- Execution Speed
- Program Size
- Message-Passing Overhead
- Program Complexity (in overuse of inheritance)

TYPES OF INHERITANCE (USING CLASSES)

- Acquiring the properties of an existing Object into newly creating Object to overcome the redeclaration of properties in deferent classes.
- The following are the types of Inheritance using classes:



TYPES OF INHERITANCE (USING CLASSES)



MEMBER ACCESS RULES

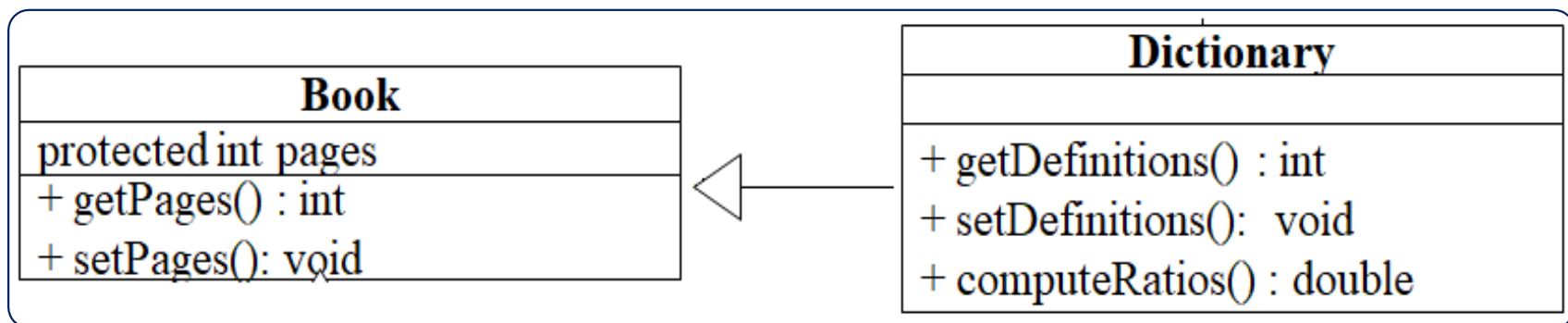
- Visibility modifiers determine which class members are accessible and which do not
- Members (variables and methods) declared with public visibility are accessible, and those with private visibility are not
- Problem: How to make class/instance variables visible only to its subclasses?
- Solution: Java provides a third visibility modifier that helps in inheritance situations: protected

MODIFIERS AND INHERITANCE (CONT.)

- Visibility Modifiers for class/interface:
- `public` : can be accessed from outside the class definition.
- `protected` : can be accessed only within the class definition in which it appears, within other classes in the same package, or within the definition of subclasses.
- `private` : can be accessed only within the class definition in which it appears.
- default-access (if omitted) features accessible from inside the current Java package

THE PROTECTED MODIFIER

- The protected visibility modifier allows a member of a base class to be accessed in the child.
- protected visibility provides more encapsulation than public does.
- protected visibility is not as tightly encapsulated as private visibility.



EXAMPLE: SUPER AND SUB CLASS

```
// SUPER CLASS A
class A {
    int i;
    void showi() {
        System.out.println("i: " + i);
    }
}

// SUB CLASS B
class B extends A {
    int j;
    void showj() {
        System.out.println("j: " + j);
    }
    void sum() {
        System.out.println("i+j: " + (i+j));
    }
}
```

EXAMPLE: TESTING CLASS

```
// TESTING CLASS
class SimpleInheritance {
    public static void main(String args[]) {
        A a = new A(); B b = new B();
        a.i = 10;
        System.out.println("Contents of a: ");
        a.showi();
        b.i = 7; b.j = 8;
        System.out.println("Contents of b: ");
        subOb.showi(); subOb.showj();
        System.out.println("Sum of I and j in b:");
        b.sum();
    }
}
```

MULTI-LEVEL CLASS HIERARCHY

```
// SUPER CLASS A
class A { void showA() { System.out.println("A"); } }
// SUB CLASS B (B - SUPER CLASS A)
class B extends A { void showB() { System.out.println("B"); } }
// SUB CLASS C (C - SUPER CLASS B)
class C extends B { void showC() { System.out.println("C"); } }
// TEST CLASS
public class Test {
    public static void main(String[] args) {
        C obj = new C();
        obj.showA();
        obj.showB();
        obj.showC();
    }
}
```


MULTI-LEVEL CLASS HIERARCHY

- Multi-Level Inheritance
 - Class A is the base (parent) class.
 - Class B extends A, so B inherits the method of A.
 - Class C extends B, so C inherits methods from both A and B.
- This creates a chain of inheritance:
 - $C \rightarrow B \rightarrow A$
- When we create an object of class C, it automatically gets access to:
 - showA() from A
 - showB() from B
 - showC() from C
- So multi-level inheritance allows a class to build on top of another class in multiple steps, each level adding new features.

“SUPER” USES

- ‘super’ is a keyword used to refer to hidden variables of super class from sub class.
 - `super.a = a;`
- It is used to call a constructor of super class from constructor of sub class which should be first statement.
 - `super(a, b);`
- It is used to call a super class method from sub class method to avoid redundancy of code
 - `super.addNumbers(a, b);`

SUPER AND HIDING

- Why is super needed to access super-class members?
- When a sub-class declares the variables or methods with the same names and types as its super-class:
- The re-declared variables /methods hide those of the super-class.

```
class A {  
    int i = 1;  
}  
  
class B extends A {  
    int i = 2;  
    B(){  
        System.out.println("i is " + i);  
    }  
}
```

EXAMPLE: SUPER AND HIDING

```
// SUPER CLASS A
class A {
    int i;
}
// SUB CLASS B
class B extends A {
    int i;
    B(int a, int b) {
        super.i = a; i = b;
    }
    void show() {
        System.out.println("i in superclass: " + super.i);
        System.out.println("i in subclass: " + i);
    }
}
```

EXAMPLE: SUPER AND HIDING

- Although the i variable in B hides the i variable in A, super allows access to the hidden variable of the super-class:

```
class UseSuper {  
    public static void main(String args[]) {  
        // SUB CLASS B OBJECT CREATION  
        B subOb = new B(1, 2);  
        // INVOKING SUB CLASS B METHOD show()  
        subOb.show();  
    }  
}
```

USING FINAL WITH INHERITANCE

- The **final** keyword is used to declare constants which can not change their defined value.
- **final** Variables can not change their value.
- **final** Methods can not be Overridden or Overloaded.
- **final** Classes can not be extended or inherited.

PREVENTING OVERRIDING WITH FINAL

- A method declared final cannot be overridden in any sub-class:

```
// SUPER CLASS A
class A {
    final void meth() {
        System.out.println("This is a final method.");
    }
}
```

This class declaration is illegal:

```
// SUB CLASS B
class B extends A {
    void meth() {
        System.out.println("Illegal!");
    }
}
```

PREVENTING INHERITANCE WITH FINAL

- A class declared final cannot be inherited – has no sub-classes.

final class A { ... }

- This class declaration is considered illegal:

class B extends A { ... }

- Declaring a class final implicitly declares all its methods final.
- It is illegal to declare a class as both abstract and final.

POLYMORPHISM

- Polymorphism is one of the three pillars of object-orientation.
- Polymorphism: many different (poly) forms of objects that share a common interface respond differently when a method of that interface is invoked:
 1. A super-class defines the common interface
 2. A subclass has to follow this interface (inheritance), but is also permitted to provide its own implementations (overriding)
- A subclass provides specialized behaviors relying on the common elements defined by its superclass.

POLYMORPHISM

- A polymorphic reference can refer to different types of objects at different times.
 - In Java, every reference can be polymorphic except for references to base types and final classes.
- It is the type of the object being referenced, not the reference type, that determines which method is invoked.
 - Polymorphic references are therefore resolved at run-time, not during compilation; this is called *dynamic binding*.
- Careful use of polymorphic references can lead to elegant, robust software designs.

METHOD OVERRIDING

- When a method of a subclass has the same name and type as a method of the superclass, we say that this method is overridden.
- When an overridden method is called from within the subclass:
 1. It will always refer to the subclass method
 2. superclass methods are hidden

EXAMPLE: HIDING WITH OVERRIDING 1

```
class A {  
    int i, j;  
    A(int a, int b) {  
        i = a; j = b;  
    }  
    void show() {  
        System.out.println("i and j: " + i + " " + j);  
    }  
}
```

EXAMPLE: HIDING WITH OVERRIDING 2

```
class B extends A {  
    int k;  
    B(int a, int b, int c) {  
        super(a, b);  
        k = c;  
    }  
    @Override  
    void show() {  
        System.out.println("k: " + k);  
    }  
}
```

EXAMPLE: HIDING WITH OVERRIDING 3

- When show() is invoked on an object of type B, the version of show() defined in B is used.
- The version of show() in A is hidden through overriding.

```
class OverrideDemo {  
    public static void main(String args[]) {  
        B subOb = new B(1, 2, 3);  
        subOb.show();  
    }  
}
```

OVERLOADING VS. OVERRIDING

OVERLOADING

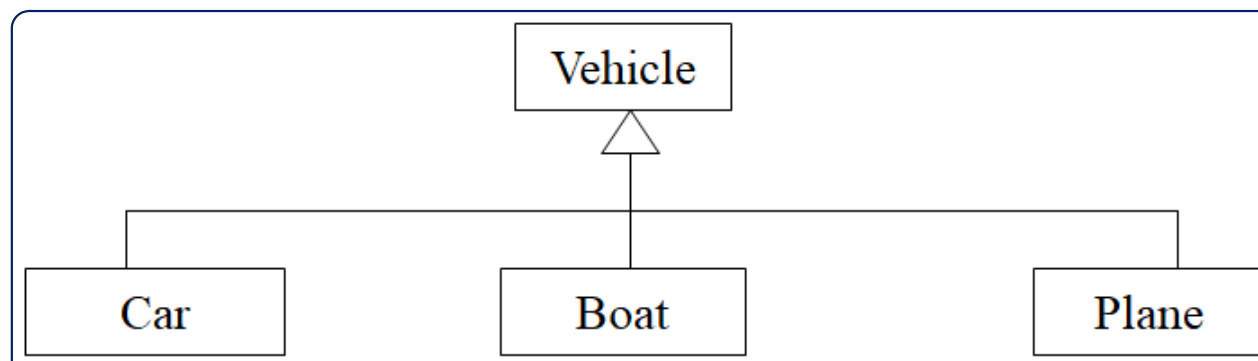
- Overloading deals with multiple methods in the same class with the same name but different signatures.
- Overloading lets you define a similar operation in different ways for different data.

OVERRIDING

- Overriding deals with two methods, one in a parent class and one in a child class, that have the same signature.
- Overriding lets you define a similar operation in different ways for different object types.

ABSTRACT CLASSES

- Java allows abstract classes
 - use the modifier `abstract` on a class header to declare an abstract class
`abstract class Vehicle{ ... }`
- An abstract class is a placeholder in a class hierarchy that represents a generic concept



ABSTRACT CLASS: EXAMPLE

- An abstract class often contains *abstract methods*, though it doesn't have to.
 - Abstract methods consist of only method *declarations*, without any method body.

```
public abstract class Vehicle {  
    String name;  
    // Method with body  
    public String getName() {  
        return name;  
    } \\ method body  
    // Method without body | Empty Method  
    abstract public void move();  
}
```

ABSTRACT CLASSES

- An abstract class often contains *abstract methods*, though it doesn't have to.
 - Abstract methods consist of only methods *declarations*, without any method body.
- The non-abstract child of an abstract class must override the abstract methods of the parent.
- An abstract class cannot be instantiated.
- The use of abstract classes is a design decision; it helps us establish common elements in a class that is too general to instantiate.

ABSTRACT METHOD

- Inheritance allows a subclass to override the methods of its superclass.
- A super-class may altogether leave the implementation details of a method and declare such a method abstract:
 - abstract type name(parameter-list);
- Two kinds of methods:
 1. concrete – may be overridden by subclasses
 2. abstract – must be overridden by subclasses
- It is illegal to define abstract constructors or static methods.

DEFINING A PACKAGE

- A package is both a naming and a visibility control mechanism:
 1. divides the name space into disjoint subsets It is possible to define classes within a package that are not accessible by code outside the package.
 2. controls the visibility of classes and their members It is possible to define class members that are only exposed to other members of the same package.
- Same-package classes may have an intimate knowledge of each other, but not expose that knowledge to other packages

CREATING A PACKAGE

- A package statement inserted as the first line of the source file:

```
package myPackage;  
class MyClass1 { ... }  
class MyClass2 { ... }
```
- means that all classes in this file belong to the **myPackage** package.
- The package statement creates a namespace where such classes are stored.
- When the package statement is omitted, class names are put into the default package, which has no name.

MULTIPLE SOURCE FILES

- Other files may include the same package instruction:
 1. package **myPackage**;
 class MyClass1 { ... }
 class MyClass2 { ... }
 2. package **myPackage**;
 class MyClass3{ ... }
- A package may be distributed through several source files.

PACKAGES AND DIRECTORIES

- Java uses file system directories to store packages.
- Consider the Java source file:

```
package myPackage;  
class MyClass1 { ... }  
class MyClass2 { ... }
```
- The byte code files **MyClass1.class** and **MyClass2.class** must be stored in a directory **myPackage**.
- Case is significant! Directory names must match package names exactly.

PACKAGE HIERARCHY

- To create a package hierarchy, separate each package name with a dot:
package **myPackage1.myPackage2.myPackage3;**
- A package hierarchy must be stored accordingly in the file system:
 1. **Unix** myPackage1/myPackage2/myPackage3
 2. **Windows** myPackage1\myPackage2\myPackage3
 3. **Macintosh** myPackage1:myPackage2:myPackage3
- You cannot rename a package without renaming its directory!

ACCESSING A PACKAGE

- As packages are stored in directories, how does the Java run-time system know where to look for packages?
- Two ways:
 1. The current directory is the default start point - if packages are stored in the current directory or sub-directories, they will be found.
 2. Specify a directory path or paths by setting the CLASSPATH environment variable.

CLASSPATH VARIABLE

- CLASSPATH - environment variable that points to the root directory of the system's package hierarchy.
- Several root directories may be specified in CLASSPATH,
- e.g. the current directory and the C:\Users\shafa\Desktop\CS305PC OOP_JAVA> directory:

Command Prompt

```
C:\Users\shafa\Desktop\CS305PC OOP_JAVA>
```

- Java will search for the required packages by looking up subsequent directories described in the CLASSPATH variable.

FINDING PACKAGES

- Consider this package statement:
`package myPackage;`
- In order for a program to find myPackage, one of the following must be true:
 1. program is executed from the directory immediately above myPackage (the parent of myPackage directory)
 2. CLASSPATH must be set to include the path to myPackage

EXAMPLE: PACKAGE

```
package MyPack;

class Balance {
    String name;
    double bal;
    Balance(String n, double b) {
        name = n; bal = b;
    }
    void show() {
        if (bal<0) System.out.print("----> ");
        System.out.println(name + ": $" + bal);
    }
}
```

EXAMPLE: PACKAGE

```
class AccountBalance {  
    public static void main(String args[]) {  
        Balance current[] = new Balance[3];  
        current[0] = new Balance("K. J. Fielding", 123.23);  
        current[1] = new Balance("Will Tell", 157.02);  
        current[2] = new Balance("Tom Jackson", -12.33);  
        for (int i=0; i<3; i++)  
            current[i].show();  
    }  
}
```

EXAMPLE: PACKAGE

- Save, compile and execute:
 1. **call** the file AccountBalance.java
 2. **save** the file in the directory **MyPack**
 3. **Compile:** AccountBalance.class should be also in **MyPack**.
 4. set access to **MyPack** in CLASSPATH variable, or make the parent of MyPack your current directory.
 5. **run:** java MyPack.AccountBalance.
- Make sure to use the package-qualified class name.

IMPORTING OF PACKAGES

- Since classes within packages must be fully-qualified with their package names, it would be tedious to always type long dot-separated names.
- The import statement allows to use classes or whole packages directly.
- Importing of a concrete class:

```
import myPackage1.myPackage2.myClass;
```

- Importing of all classes within a package:

```
import myPackage1.myPackage2.*;
```

IMPORT STATEMENT

- The import statement occurs immediately after the package statement and before the class statement:

```
package myPackage;
```

- `import otherPackage1;otherPackage2.otherClass;`

```
class myClass { ... }
```

- The Java system accepts this import statement by default:

```
import java.lang.*;
```

- This package includes the basic language functions. Without such functions, Java is of no much use.

EXAMPLE: PACKAGES 1

- A package **MyPack** with one public class **Balance**.
- The class has two same-package variables: public constructor and a public show method.

```
package MyPack;  
  
public class Balance {  
    String name; double bal;  
    public Balance(String n, double b) {  
        name = n; bal = b;  
    }  
    public void show() {  
        if (bal < 0) System.out.print("----> ");  
        System.out.println(name + ": $" + bal);  
    }  
}
```

EXAMPLE: PACKAGES 2

- The importing code has access to the public class Balance of the MyPack package and its two public members:

```
import MyPack.*;
class TestBalance {
    public static void main(String args[]) {
        Balance test = new Balance("M. S. Khan", 99.88);
        test.show();
    }
}
```

JAVA SOURCE FILE

- Finally, a Java source file consists of:
 1. a single package instruction (optional)
 2. several import statements (optional)
 3. a single public class declaration (required)
 4. several classes private to the package (optional)
- At a minimum, a file contains a single public class declaration.

DIFFERENCES BETWEEN CLASSES AND INTERFACES

- Interfaces are syntactically similar to classes, but they lack instance variables, and their methods are declared without any body.
- One class can implement any number of interfaces.
- Interfaces are designed to support dynamic method resolution at run time.
- Interface is a little bit like a class... but interface lacks instance variables....that's you can't create an object for it.....
- Interfaces are developed to support multiple inheritance...
- The methods present in interfaces are pure abstract..

DIFFERENCES BETWEEN CLASSES AND INTERFACES

- The access specifiers public, private, and protected are possible with classes, but the interface uses only one specifier, public.....
- interfaces contain only the method declarations... no definitions.....
- An interface defines which method a class has to implement. This is way - if you want to call a method defined by an interface - you don't need to know the exact class type of an object, you only need to know that it implements a specific interface.
- Another important point about interfaces is that a class can implement multiple interfaces.

DEFINING AN INTERFACE

- Using interface, we specify what a class must do, but not how it does this.
- An interface is syntactically similar to a class, but it lacks instance variables and its methods are declared without any body.
- An interface is defined with an interface keyword.

DEFINING AN INTERFACE

- An interface declaration consists of modifiers, the keyword `interface`, the interface name, a comma-separated list of parent interfaces (if any), and the interface body. Example provided at the end.
- The public access specifier indicates that the interface can be used by any class in any package. If you do not specify that the interface is public, your interface will be accessible only to classes defined in the same package as the interface.
- An interface can extend other interfaces, just as a class can extend or subclass another class. However, whereas a class can extend only one other class, an interface can extend any number of interfaces. The interface declaration includes a comma-separated list of all the interfaces that it extends.

```
public interface GroupedInterface extends Interface1, Interface2, Interface3 {  
    void doSomething (int i, double x);  
    int doSomethingElse(String s);  
}
```

IMPLEMENTING INTERFACE

- General format:

```
access interface name {  
    type method-name1(parameter-list);  
    type method-name2(parameter-list);  
    ...  
    type var-name1 = value1;  
    type var-nameM = valueM;  
    ...  
}
```


INTERFACE ACCESS SPECIFIERS

- Two types of access:
 - 1) public – interface may be used anywhere in a program
 - 2) default – interface may be used in the current package only
- Interface methods have no bodies – they end with the semicolon after the parameter list.
- They are essentially abstract methods.
- An interface may include variables, but they must be final, static and initialized with a constant value.
- In a public interface, all members are implicitly public.

INTERFACE IMPLEMENTATION

- A class implements an interface if it provides a complete set of methods defined by this interface.
 - 1) any number of classes may implement an interface
 - 2) one class may implement any number of interfaces
- Each class is free to determine the details of its implementation.
- Implementation relation is written with the implements keyword.

IMPLEMENTATION FORMAT

- General format of a class that includes the implements clause:
- Syntax:

access class name extends super-class implements
interface1, interface2, ..., interfaceN { ... }
- Access is public or default.

```
public class SubClass extends SuperClass implements interface1, interface2..., interfaceN{  
    .....  
}
```

IMPLEMENTATION COMMENTS

- If a class implements several interfaces, they are separated with a comma.
- If a class implements two interfaces that declare the same method, the same method will be used by the clients of either interface.
- The methods that implement an interface must be declared public.
- The type signature of the implementing method must match exactly the type signature specified in the interface definition.

EXAMPLE: INTERFACE

```
//Declaration of the Callback interface:
interface Callback {
    void callback(int param);
}

//Client class implements the Callback interface:
class Client implements Callback {
    public void callback(int p) {
        System.out.println("callback called with " + p);
    }
}
```

MORE METHODS IN IMPLEMENTATION

- An implementing class may also declare its own methods:

```
class Client implements Callback {  
    public void callback(int p) {  
        System.out.println("callback called with " + p);  
    }  
    void nonIfaceMeth() {  
        System.out.println("Classes that implement " +  
            "interfaces may also define " +  
            "other members, too.");  
    }  
}
```

APPLYING INTERFACES

- A Java *interface* declares a set of method signatures, i.e., says what behavior exists. It does not say how the behavior is implemented, i.e., does not give code for the methods.
- • Does not describe any state (but may include “final” constants).
- A concrete class that implements an interface Contains “implements *InterfaceName*” in the class declaration.
- Must provide implementations (either directly or inherited from a superclass) of all methods declared in the interface.
- An abstract class can also implement an interface.
- Can optionally have implementations of some or all interface methods.

APPLYING INTERFACES

- Interfaces and Extends both describe an “is-a” relation.
- If B *implements* interface A, then B inherits the (abstract) method signatures in A.
- If B *extends* class A, then B inherits everything in A, which can include method code and instance variables as well as abstract method signatures.
- Inheritance is sometimes used to talk about the superclass/subclass “extends” relation only.

VARIABLES IN INTERFACE

- Variables declared in an interface must be constants.
- A technique to import shared constants into multiple classes:
 1. declare an interface with variables initialized to the desired values.
 2. include that interface in a class through implementation.
- As no methods are included in the interface, the class does not implement anything except importing the variables as constants.

EXAMPLE: INTERFACE VARIABLES 1

An interface with constant values:

```
import java.util.Random;
interface SharedConstants {
    int NO = 0;          int YES = 1;
    int MAYBE = 2;       int LATER = 3;
    int SOON = 4;        int NEVER = 5;
}
```

EXAMPLE: INTERFACE VARIABLES 2

- Question implements SharedConstants, including all its constants.
- Which constant is returned depends on the generated random number.

```
class Question implements SharedConstants {  
    Random rand = new Random();  
    int ask() {  
        int prob = (int) (100 * rand.nextDouble());  
        if (prob < 30) return NO;  
        else if (prob < 60) return YES;  
        else if (prob < 75) return LATER;  
        else if (prob < 98) return SOON;  
        else return NEVER;  
    }  
}
```

EXAMPLE: INTERFACE VARIABLES 3

- AskMe includes all shared constants in the same way, using them to display the result, depending on the value received:

```
class AskMe implements SharedConstants {  
    static void answer(int result) {  
        switch(result) {  
            case NO: System.out.println("No"); break;  
            case YES: System.out.println("Yes"); break;  
            case MAYBE: System.out.println("Maybe"); break;  
            case LATER: System.out.println("Later"); break;  
            case SOON: System.out.println("Soon"); break;  
            case NEVER: System.out.println("Never"); break;  
        }  
    }  
}
```

EXAMPLE: INTERFACE VARIABLES 4

- The testing function relies on the fact that both ask() and answer() methods, defined in different classes, rely on the same constants:

```
public class MainClass{  
    public static void main(String args[]) {  
        Question q = new Question();  
        answer(q.ask());  
        answer(q.ask());  
        answer(q.ask());  
        answer(q.ask());  
    }  
}
```

EXTENDING INTERFACES

- One interface may inherit another interface.
- The inheritance syntax is the same for classes and interfaces.

```
interface MyInterface1 {  
    void myMethod1(...) ;  
}  
  
interface MyInterface2 extends MyInterface1 {  
    void myMethod2(...) ;  
}
```

- When a class implements an interface that inherits another interface, it must provide implementations for all methods defined within the interface inheritance chain.

EXAMPLE: INTERFACE INHERITANCE 1

- Consider interfaces A and B.

```
interface A {  
    void meth1();  
    void meth2();  
}  
  
// B extends A:  
interface B extends A {  
    void meth3();  
}
```

EXAMPLE: INTERFACE INHERITANCE 2

- MyClass must implement all of A and B methods:

```
class MyClass implements B {  
    public void meth1() {  
        System.out.println("Implement meth1().");  
    }  
    public void meth2() {  
        System.out.println("Implement meth2().");  
    }  
    public void meth3() {  
        System.out.println("Implement meth3().");  
    }  
}
```


EXAMPLE: INTERFACE INHERITANCE 3

- Create a new MyClass() object, then invoke all interface methods on it:

```
class IFExtend {  
    public static void main(String arg[]) {  
        MyClass ob = new MyClass();  
        ob.meth1();  
        ob.meth2();  
        ob.meth3();  
    }  
}
```

PACKAGE JAVA.IO

- Provides for system input and output through data streams, serialization and the file system.

Interface Summary

- **DataInput**: The DataInput interface provides for reading bytes from a binary stream and reconstructing from them data in any of the Java primitive types.
- **DataOutput**: The DataOutput interface provides for converting data from any of the Java primitive types to a series of bytes and writing these bytes to a binary stream
- **Externalizable**: Only the identity of the class of an Externalizable instance is written in the serialization stream, and it is the responsibility of the class to save and restore the contents of its instances.
- **Serializable**: Serializability of a class is enabled by the class implementing the java.io.Serializable interface.

CLASSES SUMMARY

- **BufferedInputStream**: A `BufferedInputStream` adds functionality to another input stream-namely, the ability to buffer the input and to support the `mark` and `reset` methods.
- **BufferedOutputStream**: The class implements a buffered output stream.
- **BufferedReader**: Reads text from a character-input stream, buffering characters so as to provide for the efficient reading of characters, arrays, and lines.
- **BufferedWriter**: Writes text to a character-output stream, buffering characters so as to provide for the efficient writing of single characters, arrays, and strings
- **ByteArrayInputStream**: A `ByteArrayInputStream` contains an internal buffer that contains bytes that may be read from the stream.
- **ByteArrayOutputStream**: This class implements an output stream in which the data is written into a byte array.

CLASSES SUMMARY

- **CharArrayReader**: This class implements a character buffer that can be used as a character-input stream
- **CharArrayWriter**: This class implements a character buffer that can be used as an Writer
- **Console**: Methods to access the character-based console device, if any, associated with the current Java virtual machine.
- **DataInputStream**: A data input stream lets an application read primitive Java data types from an underlying input stream in a machine-independent way.
- **DataOutputStream**: A data output stream lets an application write primitive Java data types to an output stream in a portable way.
- **File**: An abstract representation of file and directory pathnames.
- **FileInputStream**: A FileInputStream obtains input bytes from a file in a file system.
- **FileOutputStream**: A file output stream is an output stream for writing data to a File or to a FileDescriptor.

CLASSES SUMMARY

- **FileReader**: Convenience class for reading character files.
- **FileWriter**: Convenience class for writing character files.
- **FilterInputStream**: A FilterInputStream contains some other input stream, which it uses as its basic source of data, possibly transforming the data along the way or providing additional functionality.
- **FilterOutputStream**: This class is the superclass of all classes that filter output streams
- **.FilterReader**: Abstract class for reading filtered character streams
- **.FilterWriter**: Abstract class for writing filtered character streams
- **.InputStream**: This abstract class is the superclass of all classes representing an input stream of bytes.
- **InputStreamReader**: An InputStreamReader is a bridge from byte streams to character streams: It reads bytes and decodes them into characters using a specified charset.

CLASSES SUMMARY

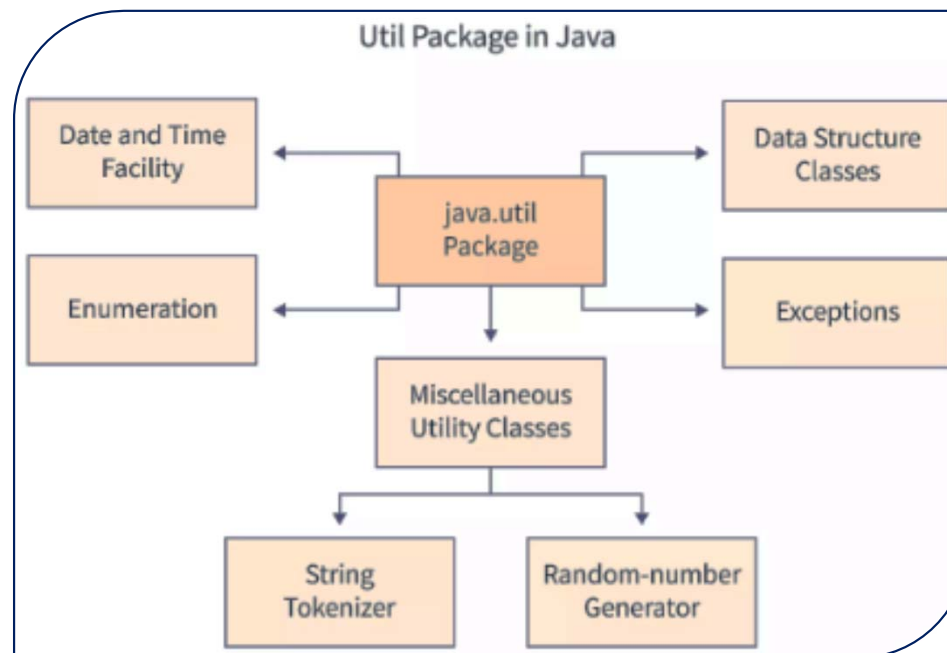
- **ObjectInputStream**: An `ObjectInputStream` deserializes primitive data and objects previously written using an `ObjectOutputStream`
- **ObjectOutputStream**: An `ObjectOutputStream` writes primitive data types and graphs of Java objects to an `OutputStream`.
- **OutputStream**: This abstract class is the superclass of all classes representing an output stream of bytes.
- **OutputStreamWriter**: An `OutputStreamWriter` is a bridge from character streams to byte streams: Characters written to it are encoded into bytes using a specified charset.
- **PrintWriter**: Prints formatted representations of objects to a text-output stream.
- **RandomAccessFile**: Instances of this class support both reading and writing to a random access file.
- **StreamTokenizer**: The `StreamTokenizer` class takes an input stream and parses it into "tokens", allowing the tokens to be read one at a time.

EXCEPTION SUMMARY

- **FileNotFoundException**: Signals that an attempt to open the file denoted by a specified pathname has failed.
- **InterruptedIOException**: Signals that an I/O operation has been interrupted
- **InvalidClassException**: Thrown when the Serialization runtime detects one of the following problems with a Class.
- **InvalidObjectException**: Indicates that one or more deserialized objects failed validation tests.
- **IOException**: Signals that an I/O exception of some sort has occurred.

PACKAGE JAVA.UTIL

- It contains the Collections Framework, legacy collection classes, the event model, date and time utilities, internationalization support, and various miscellaneous utility classes such as string tokenizers, random-number generators, and bit arrays.



INTERFACE SUMMARY

■ Collection Framework Interfaces

- **Collection<E>** – Root interface for all collection types.
- **List<E>** – Ordered collection that allows duplicates and index-based access.
- **Set<E>** – Unordered collection that does not allow duplicates.
- **SortedSet<E>** – Set whose elements are automatically sorted.
- **NavigableSet<E>** – SortedSet with navigation methods like `lower()`, `floor()`, `higher()`.
- **Queue<E>** – Collection designed for holding elements before processing (FIFO).
- **Deque<E>** – Double-ended queue allowing insertion and deletion at both ends.

INTERFACE SUMMARY

■ Map Interfaces

- `Map<K,V>` – Stores key–value pairs with unique keys.
- `SortedMap<K,V>` – Map sorted by keys automatically.
- `NavigableMap<K,V>` – `SortedMap` with navigation methods like `ceilingEntry()`, `floorEntry()`.
- `Map.Entry<K,V>` – Represents a key–value pair inside a Map.

■ Event & Observer Interfaces

- `EventListener` – Marker interface for all event-handling listeners.
- `Observer (deprecated)` – Used to implement the Observer design pattern.

INTERFACE SUMMARY

▪ **Iterators / Cursor Interfaces**

- **Iterator<E>** – Provides methods to traverse a collection.
- **ListIterator<E>** – Iterator allowing bidirectional traversal of a list.
- **Enumeration<E>** – Legacy interface to iterate elements one by one.
- **Splititerator<T>** – Iterator that supports parallel processing (Java 8).
- **PrimitiveIterator<T,T_CONS>** – Iterator for primitive values.
- **PrimitiveIterator.OfInt** – Iterator for int primitives.
- **PrimitiveIterator.OfLong** – Iterator for long primitives.
- **PrimitiveIterator.OfDouble** – Iterator for double primitives.

INTERFACE SUMMARY

■ **Utility / Functional Interfaces**

- **Comparator<T>** – Defines custom ordering logic for objects.
- **Formattable** – Allows custom formatting in Formatter.
- **RandomGenerator** – Root interface for all modern random number generators (Java 17).
- **RandomGenerator.StreamableGenerator** – RNG that supports creating stream-based random sequences.

INTERFACE SUMMARY

■ **Utility / Functional Interfaces**

- [RandomGenerator.LeapableGenerator](#) – RNG that supports leap operations for skipping sequences.
- [RandomGenerator.JumpableGenerator](#) – RNG that supports jump operations for splitting streams.
- [RandomGenerator.ArbitrarilyJumpableGenerator](#) – RNG with arbitrary jump operations.
- [RandomGenerator.SplittableGenerator](#) – RNG that can split into independent generators for parallelism.

CLASSES SUMMARY

- **AbstractCollection<E>**: This class provides a skeletal implementation of the Collection interface, to minimize the effort required to implement this interface.
- **AbstractList<E>**: This class provides a skeletal implementation of the List interface to minimize the effort required to implement this interface backed by a "random access" data store (such as an array).
- **AbstractMap<K,V>**: This class provides a skeletal implementation of the Map interface, to minimize the effort required to implement this interface.
- **AbstractQueue<E>**: This class provides skeletal implementations of some Queue operations.
- **AbstractSequentialList<E>**: This class provides a skeletal implementation of the List interface to minimize the effort required to implement this interface backed by a "sequential access" data store (such as a linked list).
- **AbstractSet<E>**: This class provides a skeletal implementation of the Set interface to minimize the effort required to implement this interface.

CLASSES SUMMARY

- ArrayList<E>: Resizable-array implementation of the List interface
- Arrays: This class contains various methods for manipulating arrays (such as sorting and searching).
- BitSet: This class implements a vector of bits that grows as needed
- Calendar: The Calendar class is an abstract class that provides methods for converting between a specific instant in time and a set of calendar fields: such as YEAR, MONTH, DAY_OF_MONTH, HOUR, and so on, and for manipulating the calendar fields, such as getting the date of the next week.
- Collections: This class consists exclusively of static methods that operate on or return collections
- Currency: Represents a currency.

CLASSES SUMMARY

- Date: The class Date represents a specific instant in time, with millisecond precision.
- Dictionary<K,V>: The Dictionary class is the abstract parent of any class, such as Hashtable, which maps keys to values.
- EventObject: The root class from which all event state objects shall be derived.
- GregorianCalendar: GregorianCalendar is a concrete subclass of Calendar and provides the standard calendar system used by most of the world.
- HashMap<K,V>: Hash table based implementation of the Map interface.
- HashSet<E>: This class implements the Set interface, backed by a hash table (actually a HashMap instance)
- .Hashtable<K,V>: This class implements a hashtable, which maps keys to values.

CLASSES SUMMARY

- **LinkedList<E>**: Linked list implementation of the List interface
- **Locale**: A Locale object represents a specific geographical, political, or cultural region.
- **Observable**: This class represents an observable object, or "data" in the model-view paradigm
- **Properties**: The Properties class represents a persistent set of properties.
- **Random**: An instance of this class is used to generate a stream of pseudorandom numbers.
- **ResourceBundle**: Resource bundles contain locale-specific objects.
- **SimpleTimeZone**: SimpleTimeZone is a concrete subclass of TimeZone that represents a time zone for use with a Gregorian calendar.

CLASSES SUMMARY

- **Stack<E>**: The Stack class represents a last-in-first-out (LIFO) stack of objects.
- **StringTokenizer**: The string tokenizer class allows an application to break a string into tokens.
- **TimeZone**: TimeZone represents a time zone offset, and also figures out daylight savings.
- **TreeMap<K,V>**: A Red-Black tree based **NavigableMap** implementation.
- **TreeSet<E>**: A **NavigableSet** implementation based on a **TreeMap**. **UUID**: A class that represents an immutable universally unique identifier (UUID).
- **Vector<E>**: The Vector class implements a growable array of objects

EXCEPTION SUMMARY

- **EmptyStackException**: Thrown by methods in the Stack class to indicate that the stack is empty.
- **InputMismatchException**: Thrown by a Scanner to indicate that the token retrieved does not match the pattern for the expected type, or that the token is out of range for the expected type.
- **NoSuchElementException**: Thrown by the nextElement method of an Enumeration to indicate that there are no more elements in the enumeration.
- **TooManyListenersException**: The TooManyListenersException is used as part of the Java Event model to annotate and implement a unicast special case of a multicast Event Source.
- **UnknownFormatConversionException**: Unchecked exception thrown when an unknown conversion is given.